

rjson: расширение JSON метками и ссылками

Кабилов Рустам, Кабилов Руслан, Заходов Егор
МФТИ, 2026

1 Постановка

Формат rjson расширяет JSON двумя конструкциями:

- метка `ident@value` — даёт значению имя;
- ссылка `@ident` — в позиции значения раскрывается в значение соответствующей метки.

Дополнительно разрешены неэкранированные ключи объектов (`name:` вместо `"name":`). Компилятор разрешает все ссылки и выводит чистый JSON.

2 Грамматика (PEG)

Используем формализм PEG: / — упорядоченный выбор, ?, *, + — кванторы.

```
Value      <- _ (IdentStart / Ref / Object / Array / String / Number) _
IdentStart <- Ident ("@" Value / Keyword)
Ref        <- "@" Ident
Object     <- "{" (Pair ("," Pair)*)? "}"
Pair       <- (String / Ident) ":" Value
Array      <- "[" (Value ("," Value)*)? "]"
Ident      <- [a-zA-Z_] [a-zA-Z_0-9]*
Keyword    <- "true" / "false" / "null"
String     <- '"' (Escape / [^"\\])* '"'
Number     <- "-"? [0-9]+ ( "." [0-9]+ )? ([eE] [+-]? [0-9]+)?
_          <- (whitespace)*
```

3 Парсер

Реализован на библиотеке Parsec в монадическо-аппликативном стиле, де можно — аппликативно, где нужна зависимость — через `do`.

Каждая лексема съедает хвостовые пробелы.

Сложность представляют идентификаторы: они начинают сразу три конструкции — метку (`ident@...`), ключ объекта (`ident:...`) и ключевое слово (`true`, `false`, `null`). Различение по контексту:

- внутри `Value` мы парсим идентификатор и затем смотрим вперёд: если идёт `@`, это метка; иначе — ключевое слово (если совпадает) или ошибка;
- внутри `Pair` за идентификатором ожидается `:`.

Такой алгоритм даёт однозначный детерминированный разбор без комбинатора `try`.

4 Разрешение ссылок

Двухпроходный алгоритм.

Проход 1. Сбор таблицы меток. Обход дерева в глубину, накапливаем `Map String Value`: для каждого узла `VLabel n inner` проверяем, не определена ли уже метка n . Если определена — ошибка `Dup`. Иначе кладём $n \mapsto inner$ и рекурсивно собираем метки из `inner`.

Поскольку таблица собирается *до* подстановки, ссылка может встречаться раньше определения метки (*forward reference*).

Проход 2. Подстановка. Обход дерева с состоянием — список `path` активно разворачиваемых имён:

- `VLabel _ inner` — снимаем обёртку;
- `VRef n` — если $n \in \text{path}$, выдаём `Cycle` с полным хронологическим путём; иначе находим n в таблице (либо `Undef`) и продолжаем разрешение с $\text{path}' = n : \text{path}$;
- контейнеры — рекурсивно с тем же путём (это ловит вложенные циклы типа `a@{x:@a}`).

Псевдокод:

```
subst table = go []
  where
    go path val = case val of
      VLabel _ inner -> go path inner
      VRef n
        | n `elem` path -> Left (Cycle (reverse path ++ [n]))
        | otherwise -> case Map.lookup n table of
          Nothing -> Left (Undef n)
          Just v' -> go (n : path) v'
      VObj ps -> VObj <$> traverse \(k,x) -> ((,) k) <$> go path x) ps
      VArr xs -> VArr <$> traverse (go path) xs
      _ -> Right val
```

5 Сериализация

Отдельный ручной эмиттер выводит чистый JSON с двухпробельными отступами; строки и числа форматируются стандартным `show`.

6 Сборка и запуск

```
cabal build
cabal run rjson < examples/universities.rjson > out.json
```

Коды возврата: 0 — успех, 1 — ошибка парсинга, 2 — ошибка разрешения.